

Syntax highlighter

Julian berges | A0178740

1. Select one programming language (the target language) that you know well (i.e., C, C++, C#, Python, etc.) and determine the different lexical categories that it uses (i.e., reserved words, operators, literals, comments, etc.)

The selected language is C++

Lexical categories	C++
Preprocessor directive	Special instructions of a c++ file starting with # Ej; #include, #define, #ifdef, #pragma once
Reserved words	int, float, if , else , return, class, while, nullptr , true, false,
Variable	Any variable with a-z , A-Z, _. Ej; myVar, _count, Health_
Float literal	A number with a decimal point or exponent, representing a real number. Ej; 3.14, 1e10, 1.5e-2
Strings literal	A sequence of characters between double quotes Ej; "Hello world" , "lalala578965rfjbwue"
Char literals	A single character between two single quotes, stored as 1 char. Ej; 'A' , '1' , '\xFF' , '\0'
Single line comment	A single line of any character starting with // and ending with a new line. Ej; // This is a comment of single line
Multi line comment	A block of character starting with /* and ending with */ Ej; /* naknsknakaknka angie

	awwf afine */
Arithmetic operator	An arithmetic operator Ej; + , - , / , * , % , ++ , - -
Assignment operator	An assignment operator Ej; = , += , -= , *= , /= , %= , &= , = , ^= , <=<= , >>=
Comparison operator	An comparison operator Ej; == , != , < , > , <= , >= , <=>
Logical operator	The logical operators Ej; && , , !
Punctuation	All the punctuations types Ej; { } [] () ; , :
Space	Space between the words, numbers, etc
Unknown	Any unknown or unmatched type.

Regex

Preprocessor directive	<code>^#[a-zA-Z_][a-zA-Z0-9_]*[^\n]*</code>
Reserved words	An alphabet of all the reserved words
Variable	<code>^[a-zA-Z_][a-zA-Z0-9_]*</code>
Float literal	<code>^\d*\.\d+([eE][+-]?\d+)?[fFIl]*</code>
Strings literal	<code>^"([^"\\] \\.)*"</code>
Char literals	<code>^'([^"\\] \\[0-9a-fA-F]{1,3} \\.)'</code>
Single line comment	<code>^\\V[^\n]*</code>
Multi line comment	<code>^\\V*[\\s\\S]*?\\V</code>
Arithmetic operator	<code>^(\\+ \\+ \\- \\- * * \\% \\%)</code>
Assignment operator	<code>^(<=> >=> \\+ \\- * \\% \\^ = =(?!=))</code>
Comparison operator	<code>^(<=> <= >= == != <>)</code>
Logical operator	<code>^(&& \\ \\ \\ !)</code>
Punctuation	<code>^[{}\\[\\]();,:.]</code>
Space	<code>^\\s+</code>
Unknown	no regex as it is unknown

Solution

The solution works in a very simple way. The tokenizer uses tail recursion to iterate over the input each call consumes one token and recurses on the remainder. It delegates to `next_token` on each step, which uses regex patterns anchored at `^` to identify what token sits at the front of the string. The only place an alphabet is involved is the `@keywords` list, which is checked after the identifier regex matches, purely to distinguish reserved words from variable names.

Time complexity

The overall time complexity of my solution is $O(n)$, this is because every token is only visited once so the only parameter is the size of the file, the characters in the file. Every call to `next_token` drops the matched characters via `drop` and recurses on what remains, so the total number of `next_token` calls is bounded by the number of tokens, which is at most n (one per character in the worst case).

Moral dilemma

The technology used in today's society has a great impact in advancing as a civilization however the more advanced we become the harder it is to maintain stability. I reflected and reached a conclusion that no matter how advanced we can become it is fundamental for humans to not become obsolete.